

PROPUESTA DE PROYECTO: OPTIMIZACIÓN DE LOGÍSTICA Y DISTRIBUCIÓN

1. Contexto del Problema

En el entorno competitivo de la logística urbana, la eficiencia en la “última milla” es crucial para la sostenibilidad financiera y la satisfacción del cliente. La empresa **Logística Distrital S.A.** enfrenta el reto de distribuir suministros tecnológicos críticos en la ciudad de Bogotá, específicamente en las localidades de Usaquén, Chapinero y Kennedy.

El problema radica en que la demanda de entregas suele superar la capacidad inmediata, y los tiempos de desplazamiento varían significativamente debido a la infraestructura vial. Se requiere un modelo que permita tomar decisiones informadas sobre la asignación de la flota vehicular para cumplir con compromisos contractuales de entrega mínima mientras se minimizan los costos operativos representados por el tiempo en ruta.

2. Planteamiento del Problema

Se deben distribuir paquetes utilizando una flota limitada de **10 camiones**. La operación se divide en tres zonas principales con características distintas de tiempo y capacidad de entrega. El objetivo principal es determinar la asignación óptima de vehículos que minimice el tiempo total de operación, enfocado para Programación Lineal, o maximice el beneficio de entregas, enfocado para Programación Dinámica, garantizando el cumplimiento de las metas mínimas por zona.

Zona (i)	Nombre	Tiempo de Viaje (t_i)	Paquetes por Vehículo (p_i)	Demanda Mínima (d_i)
1	Usaquén	60 min	15	45
2	Chapinero	45 min	10	30
3	Kennedy	90 min	20	60

3. Modelo Matemático: Método de la Gran M

Este modelo busca la optimización global bajo restricciones de desigualdad.

Variables de decisión

- x_i : número de vehículos asignados a la zona i , donde $i = 1, 2, 3$.
- a_i : variables artificiales para penalizar el incumplimiento de la demanda mínima.

Función objetivo

$$\text{mín } Z = 60x_1 + 45x_2 + 90x_3 + M(a_1 + a_2 + a_3)$$

Restricciones

$x_1 + x_2 + x_3 \leq 10$	Disponibilidad de flota
$15x_1 + a_1 \geq 45$	Demanda Zona 1
$10x_2 + a_2 \geq 30$	Demanda Zona 2
$20x_3 + a_3 \geq 60$	Demanda Zona 3
$x_i, a_i \geq 0$	No negatividad

4. Modelo Matemático: Programación Dinámica

Este modelo divide el problema en decisiones secuenciales por zonas.

Definiciones

- **Etap**a (n): representa cada zona a la que se le asignarán recursos, con $n = 1, 2, 3$.
- **Estado** (s_n): número de vehículos disponibles para ser asignados en la etapa n y etapas posteriores.
- **Decisión** (x_n): número de vehículos asignados específicamente a la zona n .

Ecuación de recurrencia

Para maximizar el beneficio total, medido como paquetes entregados:

$$f_n(s_n) = \max\{p_n(x_n) + f_{n+1}(s_n - x_n)\}$$

Sujeto a:

$$0 \leq x_n \leq s_n$$

5. Avances

Esta sección presenta la documentación de los algoritmos implementados para resolver y analizar el problema de optimización: el Método de la Gran M y los modelos de Programación Dinámica.

5.1. Método de la Gran M (MetodoDeLaGranM.py)

5.1.1. Propósito

El archivo implementa un solucionador de programación lineal usando el método simplex con la técnica de la Gran M.

5.1.2. Flujo general

- **Datos de entrada:** maximizar (booleano del sentido del problema), `norig` (variables originales $x_1, \dots, x_n$), `fostr` (cadena con la función objetivo, por ejemplo $60x_1 + 45x_2 + 90x_3$) y `constraint_lines` (restricciones en texto, por ejemplo $x_1 + x_2 + x_3 \leq 10$).
- **Parser de expresiones y restricciones:** `parse_expr` extrae coeficientes; `parse_constraint` detecta operadores ($\leq$, $\geq$, $=$) y convierte el lado derecho a fracción. También normaliza lados derechos negativos invirtiendo el signo y el operador.
- **Construcción de variables auxiliares:** agrega holguras s para $\leq$, excesos e y artificiales a para $\geq$, y artificiales a para igualdades.
- **Tabla inicial:** genera el tableau simplex con `n_vars` y `n_rows`. El vector c_j tiene los coeficientes de la función objetivo y las artificiales reciben una penalización M (`M_VALUE = 1_000_000`) para forzar su eliminación.
- **Iteraciones simplex:** calcula z_j y $c_j - z_j$, elige la columna pivote con el mayor $c_j - z_j$, aplica la regla del mínimo cociente y normaliza la fila pivote para actualizar la base.
- **Condiciones de parada:** si `best_czj` $\leq 1e-9$, se alcanza el óptimo; si no existe fila pivote válida, el problema es no acotado; si se supera `max_iter = 100`, retorna `max_iter`.
- **Comprobación de factibilidad:** si alguna variable artificial básica tiene $b > 0$, el problema se considera infactible.

5.1.3. Resultados devueltos

La función principal `solve_gran_m(...)` retorna un par compuesto por:

- **iterations:** lista de iteraciones completas con los datos del tableau, variables básicas, costos básicos, c_j , z_j , $c_j - z_j$, vector b , valor de z e información del pivote.

- **result**: diccionario con el estado final y la solución.

Campos principales de **result**:

- **status**: *optimal*, *infeasible*, *unbounded* o *max_iter*.
- **sol**: valores de las variables originales cuando hay solución óptima.
- **z_real**: valor óptimo ajustado según maximización o minimización.
- **slack_sol**: valores de holguras o excesos.
- **orig_vars**: nombres de variables originales.
- **maximizar**: booleano con el sentido del problema.

5.1.4. Interpretación de resultados

- **optimal**: el problema tiene una solución óptima factible.
- **unbounded**: se encontró una dirección de mejora infinita.
- **infeasible**: no existe una solución factible, debido a variables artificiales no eliminables.
- **max_iter**: el algoritmo no convergió antes del límite de iteraciones.

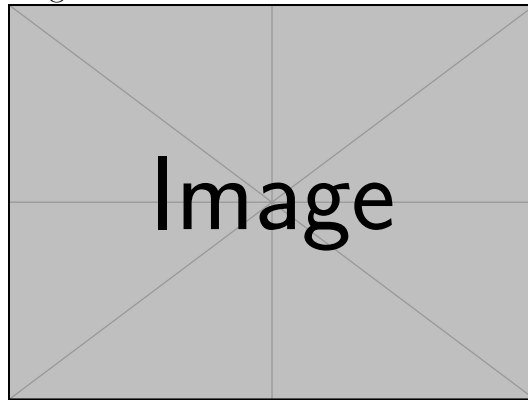
5.1.5. Puntos críticos

Las variables artificiales se penalizan con $-M$ para forzarlas fuera de la base. El método trabaja con fracciones exactas usando **fractions.Fraction**, lo que reduce errores numéricos. El código asume que la función objetivo y las restricciones son lineales en $x_1, \dots, x_n$.

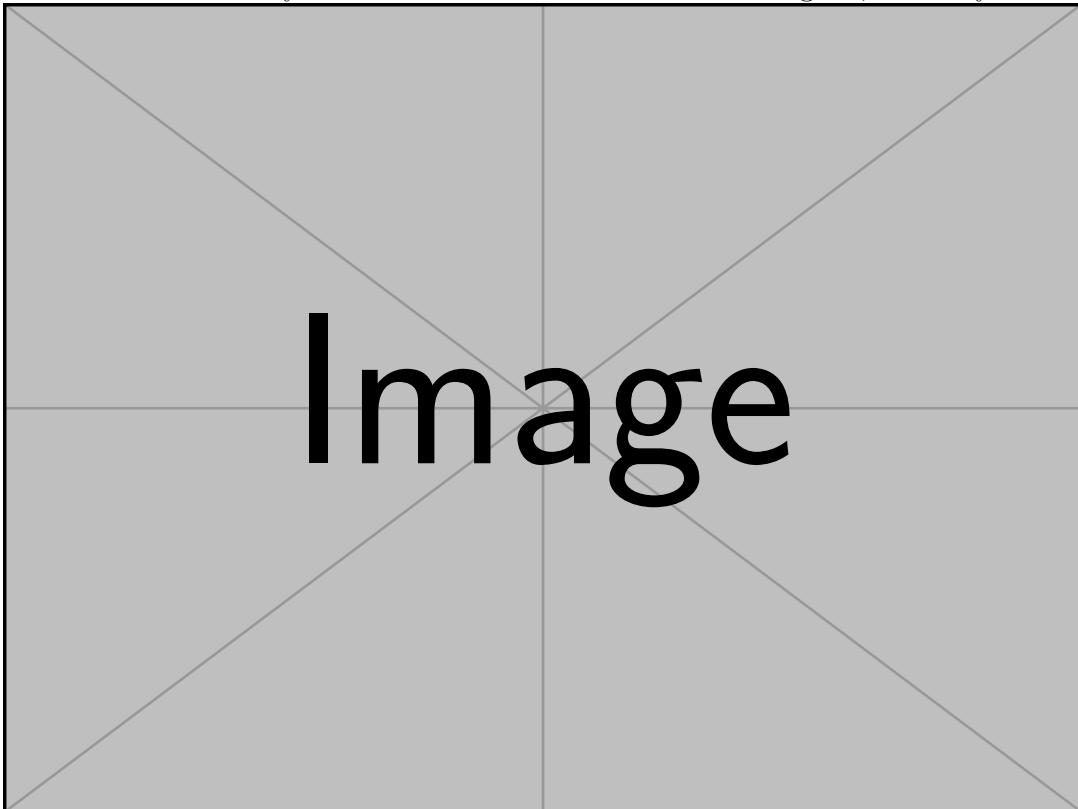
5.1.6. Muestra de prueba del software

A continuación se presenta una prueba visual del flujo del software para resolver un problema mediante el Método de la Gran M.

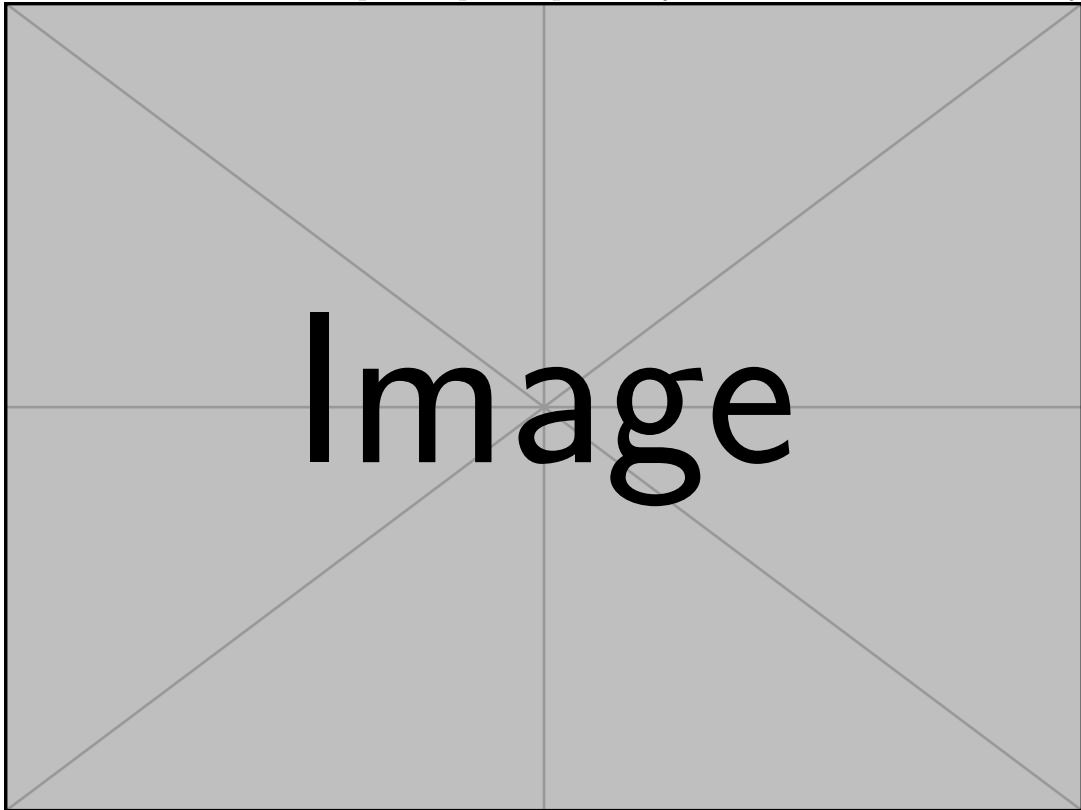
Inicio del método: se ingresan los datos iniciales del modelo de programación lineal.



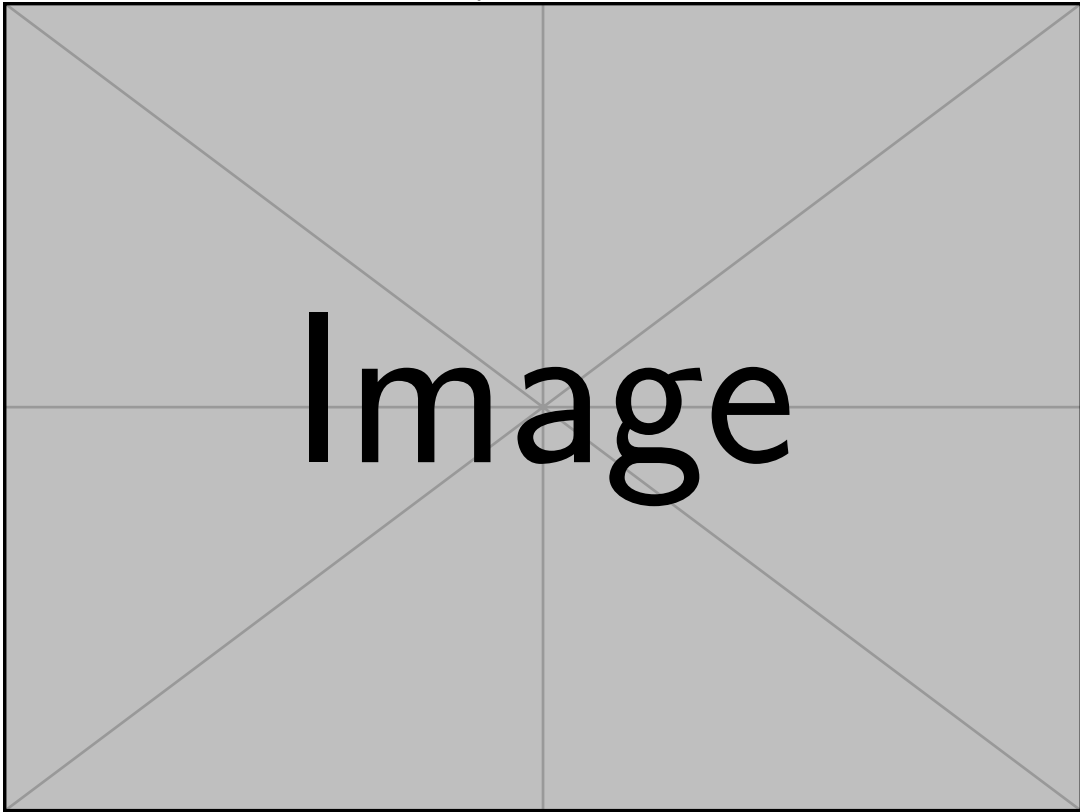
Iteración 0: se construye la tabla inicial con variables de holgura, exceso y artificiales.



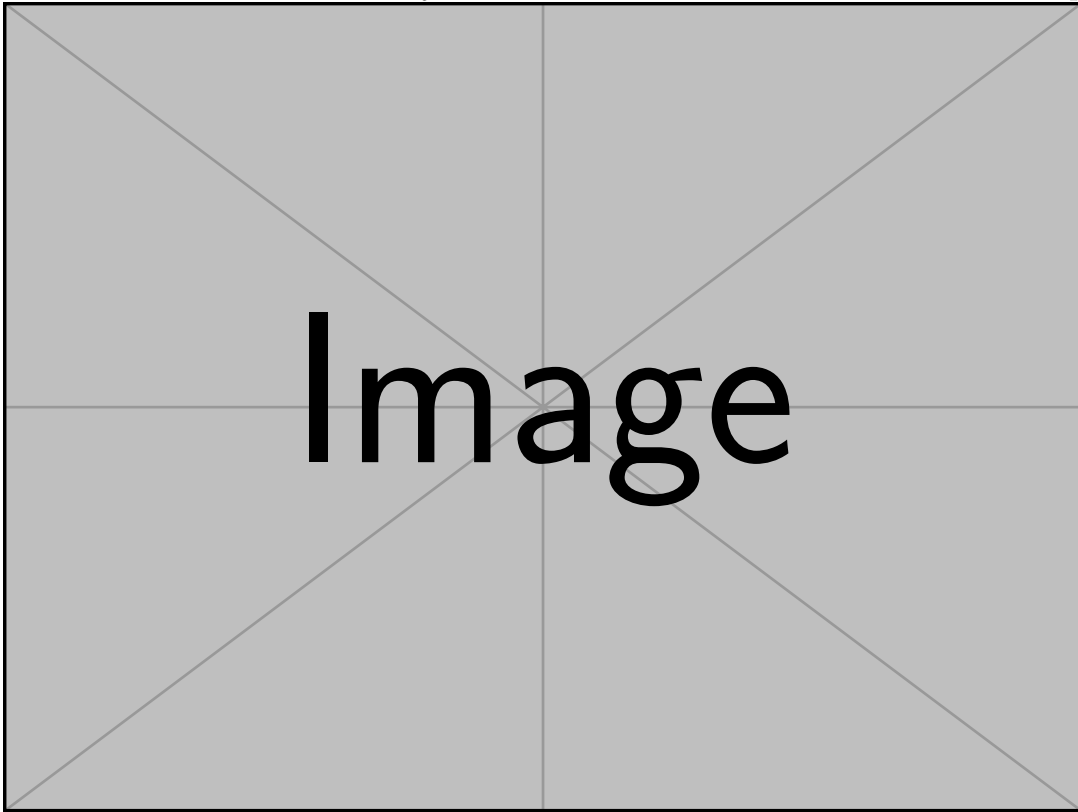
Iteración 1: se selecciona el primer pivote para mejorar el valor de la función objetivo.



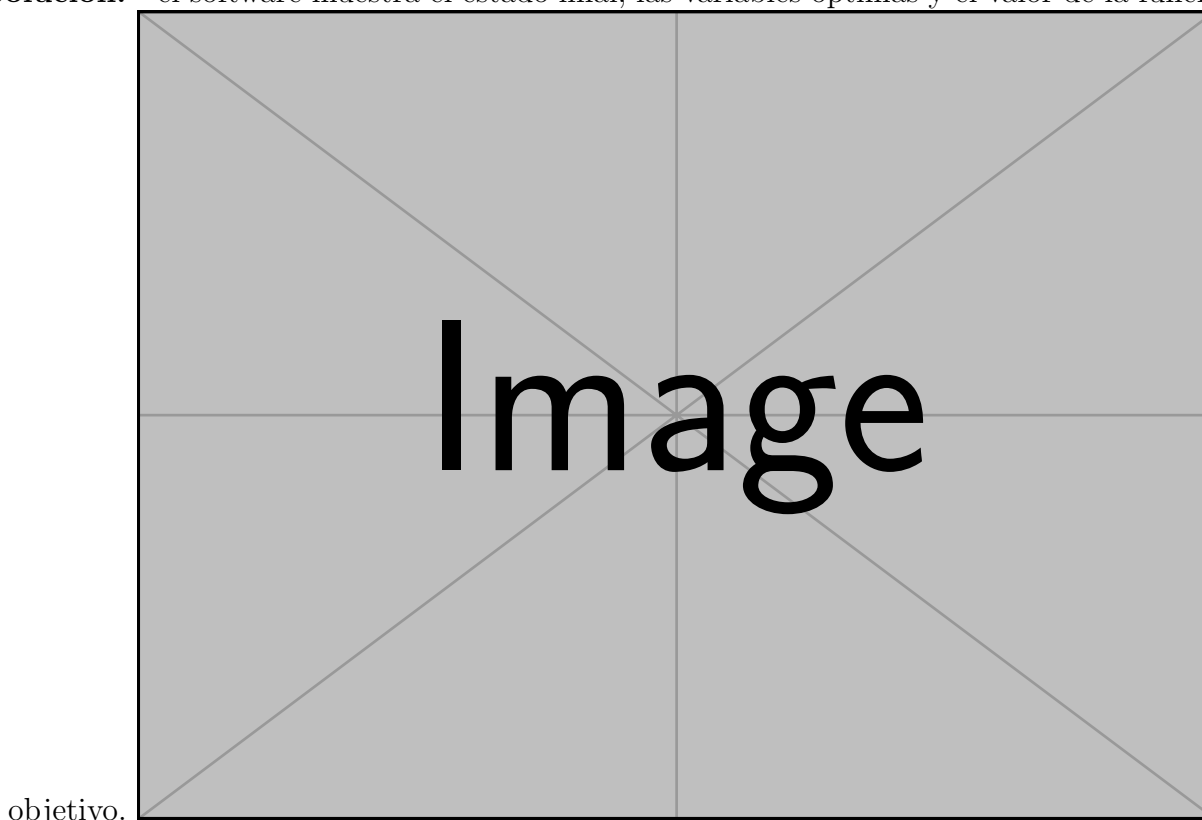
Iteración 2: se actualiza la tabla y se continúa eliminando variables artificiales.



Iteración 3: se realiza el último ajuste de la tabla antes de alcanzar la solución óptima.



Solución: el software muestra el estado final, las variables óptimas y el valor de la función



5.2. Programación Dinámica (ProgramacionDinamica.py)

5.2.1. Propósito

Este módulo ofrece varios modelos de programación dinámica para problemas de optimización discretos.

5.2.2. Estructura general

La clase `ModeloPD` define métodos estáticos para `mochila_01`, `mochila_ilimitada`, `tabla_personalizada` y `camino_dag`. Cada método construye una tabla `dp`, reconstruye la solución y genera pasos de seguimiento.

5.2.3. Método `mochila_01`

La recurrencia compara no incluir el ítem con incluirlo cuando la capacidad lo permite:

$$dp[i][w] = \text{máx}\{dp[i-1][w], dp[i-1][w-p_i] + v_i\}$$

Su salida incluye **optimo**, **seleccionados**, **peso_usado**, **dp**, **pasos** y **factible**. Soporta tipo **max** y tipo **min**, permite mínimos por ítem y reconstruye la solución comparando filas sucesivas.

5.2.4. Método mochila_ilimitada

La recurrencia evalúa reutilizar ítems mientras el peso no exceda la capacidad:

$$dp[w] = \text{máx}_i\{dp[w-p_i] + v_i\}$$

Su salida incluye **optimo**, **usados**, **dp**, **pasos** y **factible**. El estado es unidimensional, por lo que reduce memoria a $O(W)$, e incluye seguimiento **from[w]** para reconstruir los ítems usados.

5.2.5. Método tabla_personalizada

La recurrencia general por etapas puede expresarse como:

$$f[e][s] = \text{máx / mín}\{f[e-1][s'], r[e][s]\}, \quad s' \geq s$$

Su salida incluye **optimo**, **camino**, **dp** y **pasos**. Es un modelo multi-etapas que evalúa decisiones de estado y soporta restricciones de valor mínimo para cada etapa.

5.2.6. Método camino_dag

La recurrencia para rutas en grafos acíclicos dirigidos es:

$$\text{dist}[v] = \text{mín / máx}\{\text{dist}[u] + w(u, v)\}$$

Su salida incluye **optimo**, **camino**, **dp** o **dist**, y **pasos**. El método detecta ciclos, devuelve error si el grafo no es DAG y usa orden topológico para procesar cada vértice una sola vez.

5.3. Complementariedad entre ambos módulos

`MetodoDeLaGranM.py` resuelve problemas de programación lineal con variables continuas y restricciones lineales. `ProgramacionDinamica.py` resuelve problemas discretos de optimización combinatoria y de etapas. Ambos buscan optimizar una función objetivo, pero emplean estructuras matemáticas diferentes. Gran M se utiliza cuando el problema puede modelarse como programación lineal con restricciones de igualdad o desigualdad. Programación Dinámica se utiliza cuando el problema tiene estructura discreta, subproblemas superpuestos y decisiones secuenciales.

En el código actual, los dos módulos son independientes: no se invoca `MetodoDeLaGranM.solve_gran_m` desde `ProgramacionDinamica.py`, y no hay función compartida ni intercambio directo de datos. En un sistema más amplio, Gran M podría usarse para problemas de mezcla de producción, transporte o formulaciones lineales, mientras que Programación Dinámica serviría para problemas tipo mochila, rutas, planificación por etapas o decisiones discretas.

5.4. Resultados obtenidos por cada algoritmo

5.4.1. Resultados de la Gran M

Cuando la solución es óptima, el módulo devuelve los valores de $x_1, x_2, \dots, x_n$, el valor óptimo Z^* , holguras o excesos y el estado de la solución. Los estados posibles son `optimal`, `infeasible`, `unbounded` y `max_iter`.

```
{
  'status': 'optimal',
  'sol': {'x1': 12, 'x2': 4},
  'z_real': 100,
  'slack_sol': {'s1': 0, 'e2': 5},
  'orig_vars': ['x1', 'x2'],
  'maximizar': True,
}
```

5.4.2. Resultados de Programación Dinámica

Para `mochila_01`, la salida típica incluye `optimo`, `seleccionados`, `peso_usado`, `dp` y `factible`.

```
{
  'algoritmo': 'Mochila 0/1',
  'optimo': 25,
  'seleccionados': [0, 2],
  'peso_usado': 7,
  'capacidad': 10,
  'factible': True,
}
```

Para `mochila.ilimitada`, la salida típica contiene el valor óptimo para la capacidad total, la frecuencia de cada ítem y el vector de valores por capacidad.

```
{
  'algoritmo': 'Mochila Ilimitada',
  'optimo': 30,
  'usados': {1: 3, 2: 1},
  'factible': True,
}
```

Para `tabla.personalizada`, se reporta el valor final de la tabla por etapas, el camino de estados y la matriz de resultados parciales.

```
{
  'algoritmo': 'Tabla Personalizada',
  'optimo': 42,
  'camino': [2, 1, 0],
  'factible': True,
}
```

Para `camino.dag`, se reporta el costo o distancia óptima, la ruta desde el origen hasta el destino y el vector de distancias calculadas.

```
{
  'algoritmo': 'Camino en DAG',
  'optimo': 12,
  'camino': [0, 3, 5],
  'factible': True,
}
```

5.5. Conclusión de avances

`MetodoDeLaGranM.py` implementa un método robusto de programación lineal con tablas simplex y variables artificiales. `ProgramacionDinamica.py` implementa variantes de programación dinámica orientadas a mochila, tablas de etapas y rutas en grafos acíclicos. Aunque no están integrados en el código actual, ambos módulos representan enfoques complementarios de optimización: uno continuo-lineal y otro discreto-secuencial. La documentación de resultados debe orientarse a los campos estructurados que cada método retorna, ya que esa es la forma en que la aplicación puede presentar soluciones al usuario.